SCE 672: Advanced Robotics

Term Project Assessment 2: Differential Flatness Pursuit and Motor Voltage Dynamics

Instructor: Prof. Aamer Iqbal Bhatti **Marks:** 10

Institution: King Fahd University of Petroleum & Minerals **Due:** October 21st, 2026 11:59PM

Submission Instructions

Submit your assignment via the course portal before the deadline. Your submission consists of:

1. A single **PDF report** typeset in LaTeX containing your analytical steps, completed telemetry table, and execution plots.
2. A **ZIP archive / Private GitHub repository** containing your working ROS 2 package (`rm_flatness_nav`) and the exported `motor_voltage_log.csv` telemetry file.

Honor Code

- **Independent Work:** High-level architecture can be discussed with peers, but each student must write their own code, run their own robot hardware trials, and draft their own report.
- **Confidentiality:** If you use GitHub/GitLab, keep the repository strictly **Private**.

Mission Scenario

In this assessment, your RoboMaster EP will locate, align with, and track a user-designated AprilTag (T_1, T_2, or T_3) placed flat on the arena floor 1.5 m–1.8 m in front of the robot.

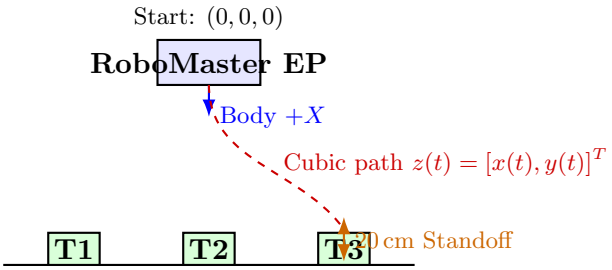


Figure 1: Arena layout: Autonomous pursuit of Target Tag T_3.

The mission requires three behaviors:

1. **Search & Align:** If the tag is hidden, rotate in place to sweep and search. When spotted, turn in place to center the tag in the camera view.
2. **Flatness Pursuit:** Generate and follow a smooth cubic trajectory in flat space to cruise toward the tag.
3. **Precision Halt & Voltage Logging:** Brake smoothly to a complete stop at an exact 20 **cm standoff distance**, while logging the real-time voltages (V_1, V_2, V_3, V_4) of all four chassis motors every second to terminal and CSV.

—

Problem 1: Visual Search, In-Place Centering & Standoff Guard (Marks: 3.0)

(i) **Downward Camera Setup:** Because tags are on the floor, the camera must be tilted downward. In your driver node, initialize the arm to ($X = 105\,\text{mm}, Y = 30\,\text{mm}$). Explain why this specific pose tilts the camera down toward the ground without blocking the front Time-of-Flight (ToF) sensor beam.

(ii) **Search and Centering Finite State Machine:** Implement a supervisory state machine in your planner node:

- `SEARCHING`: If the target tag is not visible ($\Delta t > 0.45\,\text{s}$), set forward speed to zero ($v = 0$) and slowly rotate in place ($\omega_z = \pm 0.30\,\text{rad/s}$) toward the direction where the tag was last seen.
- `ALIGNING`: When the tag is visible, rotate in place ($v = 0$) until the tag bearing error is within $\pm 10^\circ$ ($|\theta_{\text{bearing}}| \leq 0.18\,\text{rad}$).
- `CHASING`: Once centered, initiate forward pursuit along the planned trajectory.

Explain why aligning the tag in place *before* translating forward prevents trajectory drift and camera loss.

(iii) **Standoff Braking:** Implement an active brake ($v = 0, \omega = 0$) that engages immediately when either the camera distance drops to $d \leq 0.20\,\text{m}$ (20 cm) or the front ToF sensor reads $\leq 0.20\,\text{m}$.

Implementation Hint (Preventing Video Stalls): In your camera grab thread, add a small sleep (`time.sleep(0.010)`) between `cap.grab()` calls. This prevents the FFmpeg `av_frame_get_buffer` memory exhaustion error by allowing the decoder to recycle video buffers properly.

—

Problem 2: Differential Flatness Trajectory Generation (Marks: 4.0)

In **Lectures 5B and 6A**, it was shown that mobile robots can plan smooth paths without numerical optimizers by exploiting **differential flatness**.

(i) **Differential Flatness Inversion (Lecture 6A Slides 10–11):** The flat output is the position $z(t) = [x(t), y(t)]^T$. Given a smooth curve $x(t), y(t)$ with analytical derivatives $(\dot{x}, \ddot{x}, \dot{y}, \ddot{y})$, state the algebraic formulas used to recover linear speed $v(t)$ and angular turn rate $\omega(t)$:

$$v(t) = \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}, \quad \omega(t) = \frac{\dot{x}(t)\ddot{y}(t) - \dot{y}(t)\ddot{x}(t)}{\dot{x}(t)^2 + \dot{y}(t)^2} \tag{1}$$

(ii) **Cubic Polynomial Matrix Solution (Lecture 5B Slide 17):** Represent each component as a cubic curve $z_j(t) = \alpha_0^{[j]} + \alpha_1^{[j]} t + \alpha_2^{[j]} t^2 + \alpha_3^{[j]} t^3$ ($j \in \{x, y\}$) over horizon T. The boundary conditions satisfy:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} \alpha_0^{[j]} \\ \alpha_1^{[j]} \\ \alpha_2^{[j]} \\ \alpha_3^{[j]} \end{bmatrix} = \begin{bmatrix} z_j(0) \\ \dot{z}_j(0) \\ z_j(T) \\ \dot{z}_j(T) \end{bmatrix} \tag{2}$$

Write a Python function `solve_flat_polynomial(z0, zdot0, zf, zdotf, T)` using `numpy.linalg.solve` to find the polynomial coefficients $\boldsymbol{\alpha}_x$ and $\boldsymbol{\alpha}_y$. For your setup, use:

- Initial boundary: $x(0) = 0, \dot{x}(0) = 0.06\,\text{m/s}$; $y(0) = 0, \dot{y}(0) = 0$.
- Final boundary: $x(T) = d_{\text{tag}} - 0.20\,\text{m}, \dot{x}(T) = 0$; $y(T) = 0, \dot{y}(T) = 0$.

(iii) **Time Horizon Scaling (Lecture 5B Slide 20):** To keep forward cruising speed below $v_{\max} = 0.16\,\text{m/s}$ without speeding or crashing, compute T dynamically using initial tag distance d_0:

$$T = \max\left(4.0,\ \frac{1.6 \cdot (d_0 - 0.20)}{v_{\max}}\right) \tag{3}$$

—

Problem 3: Motor Voltage Telemetry and Dynamics Logging (Marks: 3.0)

In **Lecture 5A Slide 11**, it was emphasized that velocity is not an input: physical actuators are driven by motor armature voltages.

(i) **Kinematics to Wheel Rates (Lecture 5A Slide 10):** For the RoboMaster EP ($r = 0.05\,\text{m}$ wheel radius, $L_c = 0.20\,\text{m}$ chassis radius), map linear speed v_x and yaw rate ω_z to the four wheel angular speeds $(\omega_1, \omega_2, \omega_3, \omega_4)$:

$$\omega_1 = \omega_{FR} = \frac{v_x + L_c\omega_z}{r}, \quad \omega_2 = \omega_{FL} = \frac{v_x - L_c\omega_z}{r}, \quad \omega_3 = \omega_{RL} = \omega_2, \quad \omega_4 = \omega_{RR} = \omega_1 \tag{4}$$

(ii) **Motor Armature Voltage Model:** Model the terminal voltage of each motor accounting for back-EMF ($K_v = 0.38\,\text{V}\cdot\text{s/rad}$) and acceleration inertia ($K_a = 0.06\,\text{V}\cdot\text{s}^2/\text{rad}$):

$$V_i(t) = K_v\omega_i(t) + K_a\left(\frac{\omega_i(t) - \omega_i(t-\Delta t)}{\Delta t}\right), \quad i \in \{1,2,3,4\} \tag{5}$$

(iii) **Live Table and CSV Logging:** Deploy your ROS 2 system targeting **Tag 3**. Print and record the voltages at 1-second timestamps into Table 1 and export them to `motor_voltage_log.csv`. Confirm that voltages drop to 0.000 V once the robot halts at the 20 cm mark.

Table 1: Chassis Motor Armature Voltage Dynamics Log (Target: Tag 3)

Timestamp t (s)	V_1 **(V)**	V_2 **(V)**	V_3 **(V)**	V_4 **(V)**	**State**
0	0.000	0.000	0.000	0.000	`ALIGNING`
1	...	...	...	...	`ALIGNING / CHASING`
2	...	...	...	...	`CHASING`
3	...	...	...	...	`CHASING`
4	...	...	...	...	`CHASING`
...	...	...	...	...	...
T_{stop}	0.000	0.000	0.000	0.000	`ARRIVED (20cm)`

—

Deliverables Checklist

Ensure your submission includes:

1. **PDF Report:**
   - Equations used for flat trajectory inversion (v, ω) and the boundary matrix solution.
   - Completed Table 1 with actual experimental voltages.
   - One plot showing the profile of forward speed $v(t)$ and motor voltage $V_1(t)$ over time.
2. **Code Archive:** Your complete `rm_flatness_nav` package plus the logged `motor_voltage_log.csv` file.
3. **Demonstration Video Link (≤ 45 seconds):** Showing the robot (a) sweeping in place when the tag is hidden, (b) centering when Tag 3 is spotted, and (c) cruising forward and stopping at 20 cm.